Orvexa Tech
Orvexa Cloud Platform

What You Need To Do

Step-by-step operator checklist to take Orvexa Cloud live

A click-by-click guide for GitHub, MongoDB Atlas, Render and Cloudflare. Finish every staging step and the failure drills before you touch production. Do the phases in order — later phases need values created earlier.

> **Hard rule.** Staging first. Production second. Never open Atlas to `0.0.0.0/0`. Never put JWT, Razorpay or Resend keys in GitHub Actions. Never seed the production database.

Version 1.0 · 15 October 2026 · Companion to `docs/PRODUCTION_DEPLOYMENT.md`
Audience: platform owner / operator · Time: staging 1–2 days · production cutover ~2 hours with the owner present
Baseline cost after launch: about $70–80 / month (Atlas M10 + Render Starter + Cloudflare Workers Paid), plus seat fees

Contents

1. How to use this document
2. Accounts, cards and access you must have first
3. Phase 0 — Prepare secrets and the git repo
4. Phase 1 — GitHub organisation, branches and rules
5. Phase 2 — MongoDB Atlas (staging project)
6. Phase 3 — Render API (staging service)
7. Phase 4 — Cloudflare edge (staging hostnames)
8. Phase 5 — Connect GitHub Actions and do the first staging deploy
9. Phase 6 — Staging failure and rollback drills (mandatory)
10. Phase 7 — Production Atlas, Render and Cloudflare
11. Phase 8 — Launch-day cutover (owner present)
12. Appendix — Secret worksheet, DNS table, rollbacks, troubleshooting

1. How to use this document

Print this PDF or keep it open beside the four dashboards. Tick every box before you move on. If a verification at the end of a phase fails, stop and fix that phase — do not skip ahead.

- **You** click through GitHub, Atlas, Render and Cloudflare. The repository already contains the code, `render.yaml`, Workers, and GitHub Actions.
- **Staging** uses `*.staging.orvexacloud.in` and the free Atlas M0 cluster.
- **Production** uses `*.orvexacloud.in` and a dedicated Atlas M10 in AWS Singapore.
- Text in `monospace` is a value to type, a hostname, or a file in this repo.

> **Chicken-and-egg (Atlas ↔ Render).** Atlas must not allow the whole internet. Render will not become healthy until Atlas allows Render's Singapore outbound IPs. Create the cluster and users first with *no* open world access, create the Render service (it will fail health once), copy the outbound IPs into Atlas, then wait for health. That sequence is written into Phases 2 and 3.

What "done" looks like

- `https://api.orvexacloud.in/api/health` returns `{"ok":true,"commit":"<the git sha you just deployed>"}`
- `https://admin.orvexacloud.in` asks for a Cloudflare email PIN, then the Orvexa admin sign-in
- `https://app.orvexacloud.in` is the seller portal
- `https://lunar.orvexacloud.in` (after you provision a real store) is a storefront
- Product photos load from `https://media.orvexacloud.in`
- There is exactly one super admin, created with the bootstrap key, and that key has been deleted

2. Accounts, cards and access you must have first

Create these before Phase 0. Use a company email, not a personal Gmail, for every owner login.

Service	Plan / product	Why
GitHub	Organization on the **Team** plan (required for environment reviewers on a private repo)	Branch rules, CODEOWNERS, deploy approvals
MongoDB Atlas	Ability to create two **projects**	Isolated staging vs production
Render	Workspace that can run a Singapore **Starter** web service	Express API
Cloudflare	Domain `orvexacloud.in` on Cloudflare DNS + **Workers Paid**	Edge, shop router, R2, Access
Resend	Sending domain `mail.orvexacloud.in`	Owner emails
Razorpay	Live account (India) when you are ready to charge	Subscriptions

> **Cards.** Atlas M10 and Render Starter need a payment method. Workers Paid is a few dollars a month. Do not start production cutover on trial credits that expire that week.

People

- At least two GitHub users in team `@orvexa/platform` so production deploys cannot be a single-person push.
- The business owner available for Phase 8 (about two hours).

PHASE 0 Prepare secrets and the git repo

Do this on your laptop. Store every generated value in a password manager (1Password / Bitwarden), not in chat, not in a spreadsheet in Drive, not in this PDF.

0.1 Confirm you are in the right folder

```
cd C:\Users\Black\Downloads\orvexacloud-production-hardened\orvexacloud
dir render.yaml
dir .github\workflows
dir infra\cloudflare
```

You should see `render.yaml`, the five workflow files, and the shop-router Worker. If any are missing, stop — you are in the wrong copy of the project.

0.2 Generate secrets (run each line once per environment)

You need two independent sets: one labelled **STAGING**, one labelled **PRODUCTION**. Never reuse a production secret on staging.

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Run it until you have filled the worksheet in the appendix. Minimum set:

Name	How	Where it will live
`JWT_SECRET`	32+ hex chars (command above)	Render only
`JWT_REFRESH_SECRET`	A *different* 32+ hex value	Render only
`ADMIN_BOOTSTRAP_KEY`	Another 32+ hex value (≥ 16 chars)	Render only, deleted after first admin
`SIGNUP_SIGNING_SECRETS`	Another 32+ hex value	Render + marketing-site connector

> **Do not use** the placeholder strings from `.env.example`. The server refuses those in production.

0.3 Push this repo to the GitHub organization

1. On GitHub.com click **New organization** (or open the existing Orvexa org). Team plan.
2. Create a **private** repository, for example `orvexa/orvexacloud`.
3. From this folder:

   ```
   git remote add origin https://github.com/orvexa/orvexacloud.git
   git branch -M main
   git push -u origin main
   ```

4. Create the `staging` branch from `main` and push it:

   ```
   git checkout -b staging
   git push -u origin staging
   ```

☐ Repo is private under the org

- [ ] Branches `main` and `staging` both exist on GitHub
- [ ] Secrets worksheet started (appendix)

PHASE 1 GitHub organisation, branches and rules

Open the repository on GitHub.com. Complete every step in this phase before Atlas.

1.1 Create the review team

1. Org → **Teams** → **New team**.
2. Name: `platform`. Visibility: Visible.
3. Add at least two people (you + a second engineer or the owner's technical deputy).
4. Repo → **Settings** → **Collaborators and teams** → Add team `platform` with **Write** (or Maintain).

This matches `.github/CODEOWNERS`, which requires `@orvexa/platform` on every change. If your org handle is not `orvexa`, edit CODEOWNERS to `@YOURORG/platform` in a PR before locking the branch.

1.2 Branch rulesets (do this twice: `staging` and `main`)

1. Repo → **Settings** → **Rules** → **Rulesets** → **New ruleset** → **New branch ruleset**.
2. Name: `protect-staging` (then later `protect-main`).
3. Enforcement: **Active**.
4. Target branches: add the branch name (`staging` or `main`).
5. Tick:
   - Restrict deletions
   - Block force pushes
   - Require a pull request before merging — at least **1** approval
   - Require review from Code Owners
   - Require conversation resolution before merging
   - Require status checks to pass — after the first workflow run, add `check` and `scan`
6. Do *not* allow the bypass list except org admins, and only for a written incident.
7. Save. Repeat for `main`.

> Status checks cannot be selected until they have run once. Come back to this screen after Phase 5's first pull request and attach `Check & Audit / check` and `Secret Scan / scan`.

1.3 Environments

1. Repo → **Settings** → **Environments** → **New environment** → name `staging`. Save (reviewers optional).
2. **New environment** → name `production`.
3. On `production`: enable **Required reviewers** → add team `platform`. Deployment branches: only `main`.

Do not add application secrets here yet. That is Phase 5 (hooks and Cloudflare tokens only).

- ☐ Team `platform` exists with two members
- ☐ Rulesets active on `staging` and `main`
- ☐ Environments `staging` and `production` exist; production has a required reviewer

PHASE 2 MongoDB Atlas — staging project

Use a **new Atlas project**. Do not put staging data in a cluster you later promote to production.

2.1 Create the project and cluster

1. cloud.mongodb.com → **New Project** → name `orvexa-staging`.
2. **Build a Database** → **M0** Free → provider AWS (any region is fine for staging) → name the cluster `orvexa-staging`.
3. Skip the "create a user + allow 0.0.0.0/0" shortcut if Atlas offers it. You will create users yourself and you will **not** allow the world.

2.2 Database users (least privilege)

1. **Database Access** → **Add New Database User**.
2. User `orvexa_master`. Authentication: Password. Generate a long password; save it in the password manager as `STAGING_MONGO_MASTER_PASSWORD`.
3. Privileges: **Specific privileges** → `readWrite` on database `orvexatech_master`. Add user.
4. **Add New Database User** again. User `orvexa_tenant`. New password → `STAGING_MONGO_TENANT_PASSWORD`.
5. Privileges: `readWrite` on database name `orvexa_tenant_*` if Atlas offers a wildcard, otherwise `readWriteAnyDatabase` only if wildcard is unavailable — tighten later. Prefer a custom role limited to the `orvexa_tenant_` prefix.

2.3 Network access — leave it closed for now

1. **Network Access**. If Atlas created `0.0.0.0/0`, **delete it immediately**.
2. Leave the list empty, or add only your own office IP so you can use Compass if you need it. You will add Render IPs in Phase 3.3.

2.4 Copy the connection strings

1. Cluster → **Connect** → **Drivers** → Node.js.
2. Copy the SRV host, for example `orvexa-staging.xxxxx.mongodb.net`.
3. Build these two URIs (passwords URL-encoded if they contain `@` `/` `#`):

```
MONGODB_MASTER_URI=
mongodb+srv://orvexa_master:PASSWORD@HOST/orvexatech_master?retryWrites=true&w=majority

MONGODB_TENANT_BASE_URI=
mongodb+srv://orvexa_tenant:PASSWORD@HOST/orvexa_tenant_
```

> The tenant URI must end with the database *prefix* `orvexa_tenant_`. The API appends the store slug and calls `useDb('orvexa_tenant_' + slug, { useCache: true })`.

- ☐ Project `orvexa-staging` + M0 cluster exist
- ☐ Users `orvexa_master` and `orvexa_tenant` created; passwords stored
- ☐ No `0.0.0.0/0` entry

- [ ] Both URIs written on the staging secret worksheet

PHASE 3 Render — staging API

3.1 Create the Blueprint

1. dashboard.render.com → **New** → **Blueprint**.
2. Connect the GitHub org and the `orvexacloud` repo (read access is enough; Render must *not* auto-deploy).
3. Render reads `render.yaml`. You will see `orvexa-api-staging` and `orvexa-api-production`.
4. Apply **only** `orvexa-api-staging` now. Leave production for Phase 7.
5. Region must be **Singapore**. Plan **Starter**.

> **Auto-Deploy must be No / off.** Open the new service → Settings → Build & Deploy → Auto-Deploy → **No**. The repo already sets `autoDeploy: false`; confirm the UI matches.

3.2 Bind staging secrets

Service → **Environment**. Values already set by the blueprint stay. Fill every blank (`sync: false`) from the staging worksheet:

Key	Value
`MONGODB_MASTER_URI`	Staging master URI from Phase 2.4
`MONGODB_TENANT_BASE_URI`	Staging tenant prefix URI
`JWT_SECRET`	Staging JWT
`JWT_REFRESH_SECRET`	Different staging refresh JWT
`ADMIN_BOOTSTRAP_KEY`	Staging bootstrap key
`SIGNUP_SIGNING_SECRETS`	Staging signup HMAC
`RESEND_API_KEY` / `MAIL_FROM` / `MAIL_REPLY_TO`	Staging Resend (or leave empty until mail is verified)
`RAZORPAY_*`	Leave empty on staging unless you have test keys

Confirm these are already `production` / correct (blueprint):

- `NODE_ENV=production` (yes — even on staging; this is "run the real server", not "this is the live shop")
- `TRUST_PROXY=1`
- `PLATFORM_ROOT_DOMAIN=staging.orvexacloud.in`
- `PUBLIC_APP_URL=https://admin.staging.orvexacloud.in`
- `VITE_DEMO_MODE=false`

3.3 Copy Render outbound IPs into Atlas

1. Render service → look for **Outbound** / networking / "static outbound IP". Singapore Starter publishes the `render-prod-sg` (or current Singapore) egress list.
2. Copy every IPv4 CIDR shown.
3. Atlas project `orvexa-staging` → **Network Access** → **Add IP Address** → paste each CIDR → comment `render-prod-sg`.

4. Still no `0.0.0.0/0`.

3.4 Health check path and first manual deploy

1. Settings → Health Check Path = `/api/health`.
2. Manual **Deploy latest commit** once so the instance exists. It may flap until Atlas has the IPs (step 3.3).
3. When the service is Live, open `https://<render-hostname>.onrender.com/api/health`.
4. You want HTTP 200 and a JSON body like `{"ok":true,"commit":"…"}`. If you get 503, Atlas still cannot see Render — re-check IPs.

3.5 Create the staging deploy hook

1. Service → **Settings** → **Deploy Hook** → create one. Copy the URL (looks like `https://api.render.com/deploy/srv-…?key=…`).
2. Save it as `RENDER_STAGING_DEPLOY_HOOK` in the password manager. GitHub gets this in Phase 5 — nothing else.

Write down the Render service hostname (for the Cloudflare `api` CNAME), for example `orvexa-api-staging.onrender.com`.

- [ ] Service `orvexa-api-staging` in Singapore, Auto-Deploy **No**
- [ ] All secrets bound; JWT pair are different
- [ ] Atlas allow-list = Render Singapore IPs only
- [ ] `/api/health` returns `ok: true` on the Render URL
- [ ] Deploy hook URL stored

PHASE 4 Cloudflare — staging hostnames

This phase assumes `orvexacloud.in` is already on Cloudflare DNS. Staging lives on the `staging.orvexacloud.in` zone *or* as names under it. The simplest pattern: treat `staging.orvexacloud.in` as the staging root and put `api.`, `admin.`, `app.`, `shop.`, `media.` in front of it.

4.1 SSL / TLS

1. Cloudflare dashboard → domain → **SSL/TLS** → Overview → **Full (strict)**.
2. **Edge Certificates** → Minimum TLS Version → **1.2**.
3. Always Use HTTPS: On.

4.2 Create Cloudflare Pages projects (empty, then CI fills them)

1. **Workers & Pages** → **Create** → **Pages** → **Direct Upload** (CI will deploy; you only need the project names).
2. Create three staging projects with *exactly* these names (the workflow uses them):
   - `orvexa-admin-staging`
   - `orvexa-app-staging`
   - `orvexa-shop-staging`
3. Each project → Custom domains:
   - admin project → `admin.staging.orvexacloud.in`
   - app project → `app.staging.orvexacloud.in`
   - shop project → `shop.staging.orvexacloud.in`

4.3 DNS records

Name	Type	Content	Proxy	Worker
`api.staging`	CNAME	Render hostname from 3.5	Proxied (orange)	**None**
`admin.staging`	CNAME	Pages admin target Cloudflare shows	Proxied	**None**
`app.staging`	CNAME	Pages app target	Proxied	**None**
`shop.staging`	CNAME	Pages shop target	Proxied	**None**
`media.staging`	CNAME	R2 custom-domain target (4.5)	Proxied	**None**
`*.staging`	AAAA	`100::`	Proxied	`orvexa-shop-router-staging`

> Orange-cloud (proxied) on every record above. The AAAA `100::` is a placeholder so the Worker can answer storefront hosts. Nothing sits behind it.

4.4 Worker routes — bypass system hosts

1. Workers & Pages → the shop-router will be published by GitHub in Phase 5. After the first deploy, open it → **Triggers** / Routes.
2. Attach the Worker to `*.staging.orvexacloud.in/*`.
3. Add **Worker: None** routes (or do not attach the Worker) for: `api.staging.orvexacloud.in/*`, `admin.staging.orvexacloud.in/*`, `app.staging.orvexacloud.in/*`, `media.staging.orvexacloud.in/*`,

`www.staging.orvexacloud.in/*`, `mail.orvexacloud.in/*`.

4.5 R2 bucket (staging)

1. **R2** → **Create bucket** → name `orvexa-media-staging`.
2. Settings → **CORS policy** → paste `infra/cloudflare/r2-cors.json`, then change origins to the staging hosts (`https://app.staging.orvexacloud.in` for PUT).
3. Connect a custom domain `media.staging.orvexacloud.in`.
4. Public read: on. Client PUT: only the seller origin in CORS. No public write.

4.6 Zero Trust Access on staging admin (rehearsal for production)

1. Cloudflare One / Zero Trust → **Access** → **Applications** → **Add an application** → Self-hosted.
2. Application domain: `admin.staging.orvexacloud.in`, path `/` (the whole site).
3. Identity: One-time PIN (email).
4. Policy: Allow emails in your staff list.
5. Session duration: **8 hours**.

4.7 WAF and rate limit (can wait until production if you want a cleaner rehearsal, but do it on staging if the zone is shared)

1. Security → WAF → Managed rules: on. Bot Fight Mode: on.
2. Security → WAF → Rate limiting rules → create:
   - If incoming request URI path starts with `/api/auth/` *and* hostname is `api.staging.orvexacloud.in` (and later `api.orvexacloud.in`)
   - Count: **10 requests per 1 minute** per IP
   - Action: Block for **1 hour**

4.8 Cloudflare API token for GitHub

1. My Profile → **API Tokens** → **Create Token**.
2. Use a custom token: Account → Cloudflare Pages (Edit), Account → Workers Scripts (Edit), Account → Account Settings (Read), Zone → Zone (Read) + DNS (Edit) if you want Wrangler to touch DNS.
3. Include this account only.
4. Copy the token once. Save as `CLOUDFLARE_API_TOKEN`.
5. Workers & Pages overview → copy **Account ID** → `CLOUDFLARE_ACCOUNT_ID`.

- [ ] Full (strict) + TLS 1.2+
- [ ] Three Pages project names match the workflow
- [ ] DNS table applied; system hosts have Worker: None
- [ ] R2 bucket + CORS + media hostname
- [ ] Access PIN on staging admin, 8-hour session
- [ ] API token and account ID stored (for GitHub, not for the app)

PHASE 5 Connect GitHub Actions and first staging deploy

5.1 Put only hooks and edge credentials in GitHub

1. Repo → **Settings** → **Environments** → `staging` → **Add secret**:

Secret	Value
`RENDER_STAGING_DEPLOY_HOOK`	Hook from Phase 3.5
`CLOUDFLARE_API_TOKEN`	Token from Phase 4.8
`CLOUDFLARE_ACCOUNT_ID`	Account ID from Phase 4.8

Optional variables (same environment → Variables):

Variable	Value
`STAGING_API_URL`	`https://api.staging.orvexacloud.in`
`STAGING_WEB_URL`	`https://admin.staging.orvexacloud.in`

> If you are about to paste `JWT_SECRET` or `MONGODB_MASTER_URI` into GitHub, you are in the wrong dashboard. Those stay on Render.

5.2 Open a pull request into `staging`

1. Create a tiny branch (docs-only is fine) and open a PR **into** `staging`.
2. Wait for **Check & Audit** and **Secret Scan** to go green.
3. Go back to the branch ruleset and attach those two checks if you could not in Phase 1.
4. Get a `platform` team approval. Merge.

5.3 Watch the staging deploy

1. Actions tab → workflow **Deploy staging**.
2. It must run: Check → Secret Scan → Pages (admin/app/shop) → shop-router → Render hook with `ref=<sha>` → wait-for-commit → smoke.
3. If it asks to use the `staging` environment, approve it.

5.4 Verify by hand

```
curl -sS -D- https://api.staging.orvexacloud.in/api/health
```

Expect:

- HTTP 200
- Header `Cache-Control: no-store`
- Body `{"ok":true,"commit":"<the merged SHA>"}` — the SHA must match the merge commit on `staging`

```
curl -sS -o NUL -w "%{http_code}" https://api.staging.orvexacloud.in/api/admin/overview
```

Expect `401` (not 200). Then open `https://admin.staging.orvexacloud.in` — you should get the Cloudflare email PIN first.

5.5 Bootstrap the staging super admin (optional but useful for drills)

```
set API_BASE=https://api.staging.orvexacloud.in
set ADMIN_BOOTSTRAP_KEY=paste-staging-key
set ADMIN_EMAIL=you@orvexatech.com
set ADMIN_PASSWORD=choose-a-long-password
set CONFIRM_PRODUCTION_BOOTSTRAP=yes
npm run admin:bootstrap
```

Then delete `ADMIN_BOOTSTRAP_KEY` from the *staging* Render service if you want to rehearse that too. You can leave a staging key only if the staging database is disposable.

- [ ] GitHub staging environment has three secrets only (hook + two Cloudflare values)
- [ ] First PR into `staging` is green
- [ ] Health commit equals the merge SHA
- [ ] Unauthenticated admin API is 401
- [ ] Access PIN appears on the admin hostname

PHASE 6 Staging drills — do not skip

Production cutover is forbidden until all four drills pass. Write the date you completed each one on the last page.

Drill A — failed tests must not deploy

1. Open a PR to `staging` that deliberately breaks a unit test (or add `throw new Error('drill')` in a test).
2. Confirm **Check & Audit** fails and **Deploy staging** never starts (or stops before the hook).
3. Close the PR. Do not merge it.

Drill B — bad SHA must not take traffic

1. Note the current healthy commit from `/api/health`.
2. In Render, trigger a deploy of a commit that cannot boot (or temporarily point the hook at a nonsense ref if you are rehearsing wait-for-commit locally).
3. The wait step in Actions should time out. The previous Live instance on Render should still answer the old SHA.

Drill C — rollback.yml restores a known-good SHA

1. Actions → **Rollback** → Run workflow.
2. Target: `staging`. SHA: the last green merge SHA.
3. Approve the environment if asked.
4. When it finishes, `/api/health` must report that SHA again.

Drill D — secret scan blocks a leak

1. On a throwaway branch, add a file containing a fake AWS key in the well-known `AKIA…` shape, or a `-----BEGIN RSA PRIVATE KEY-----` block.
2. Open a PR. **Secret Scan** must fail. Delete the branch. Never merge it.

> **Only after A–D are ticked** do you start Phase 7. If any drill failed, fix the pipeline first.

- ☐ Drill A date: _____
- ☐ Drill B date: _____
- ☐ Drill C date: _____
- ☐ Drill D date: _____

PHASE 7 Production Atlas, Render and Cloudflare

Repeat Phases 2–5 with production names and **new** secrets. Do not copy staging passwords.

7.1 Atlas project `orvexa-production`

1. **New Project** → `orvexa-production`.
2. Cluster: **M10 dedicated replica set**, provider **AWS**, region **Singapore** (`ap-southeast-1` or the Atlas label that matches).
3. Users: new `orvexa_master` and `orvexa_tenant` with *new* passwords.
4. Network: empty except, in a moment, production Render Singapore IPs. **Never** `0.0.0.0/0`.
5. Build production URIs the same way as 2.4, with the production host and passwords.

7.2 Render service `orvexa-api-production`

1. Apply the remaining service from `render.yaml`, or New Blueprint → only production.
2. Singapore, Starter (upgrade later if CPU is short). Auto-Deploy **No**.
3. Bind every secret from the **production** worksheet. Set `ALLOW_SEED=no`.
4. Copy this service's outbound IPs into Atlas production. Confirm they match Singapore.
5. Health Check Path `/api/health`.
6. Create `RENDER_PRODUCTION_DEPLOY_HOOK`.
7. Do **not** run `npm run seed` against this URI. Ever.

7.3 Cloudflare production hostnames

Create Pages projects `orvexa-admin`, `orvexa-app`, `orvexa-shop`. Create R2 bucket `orvexa-media` and apply `infra/cloudflare/r2-cors.json` unchanged (PUT only from `https://app.orvexacloud.in`).

Name	Type	Content	Worker
`api`	CNAME	Production Render hostname	None
`admin`	CNAME	Pages `orvexa-admin`	None
`app`	CNAME	Pages `orvexa-app`	None
`media`	CNAME	R2 `orvexa-media`	None
`*`	AAAA	`100::`	`orvexa-shop-router`

Access application on `admin.orvexacloud.in`, email PIN, 8-hour session, staff inboxes only. Rate limit `api.orvexacloud.in/api/auth/`: 10 / minute / IP, block 1 hour.

7.4 GitHub production environment secrets

Secret	Environment
`RENDER_PRODUCTION_DEPLOY_HOOK`	`production`
`CLOUDFLARE_API_TOKEN`	`production` (same token is fine)
`CLOUDFLARE_ACCOUNT_ID`	`production`

Variables: `PRODUCTION_API_URL=https://api.orvexacloud.in`, `PRODUCTION_WEB_URL=https://admin.orvexacloud.in`.

- [ ] Production Atlas is M10 Singapore, no world-open IP
- [ ] Production Render Auto-Deploy is No; hook stored in GitHub `production` only
- [ ] Production DNS + Access + WAF + R2 CORS applied
- [ ] No production JWT/Mongo/Razorpay/Resend value exists in GitHub

PHASE 8 Launch-day cutover (owner present)

Block two hours. Have Slack/phone with the second `platform` reviewer. Have the last green `staging` SHA written down in case you abort and stay on staging.

8.1 Pre-flight (15 minutes)

- ☐ Atlas production Network Access shows only Render Singapore IPs — screenshot it
- ☐ Render production Auto-Deploy = No
- ☐ Cloudflare SSL = Full (strict)
- ☐ Access is on for `admin.orvexacloud.in`
- ☐ WAF rate limit on `/api/auth/` is live
- ☐ R2 CORS PUT origin is only `https://app.orvexacloud.in`
- ☐ `VITE_DEMO_MODE` is false in the production workflow (already in the repo)
- ☐ Owner is on the call

8.2 Merge to `main`

1. Open a PR `staging` → `main`. Title it with the date, e.g. `Release 2026-10-15`.
2. Checks green. Code Owners + second reviewer approve.
3. Merge (merge commit or squash — either is fine; copy the resulting SHA).
4. Actions → **Deploy production** starts. The second person approves the `production` environment.
5. Do not close the laptop until smoke is green.

8.3 Confirm the live SHA

```
curl -sS https://api.orvexacloud.in/api/health
```

The `commit` field must equal the merge SHA on `main`. If it does not after 10 minutes, use Rollback (appendix) — do not keep clicking Redeploy.

8.4 Create the one production super admin

```
set API_BASE=https://api.orvexacloud.in
set ADMIN_BOOTSTRAP_KEY=paste-production-key
set ADMIN_EMAIL=owner-or-you@orvexatech.com
set ADMIN_PASSWORD=long-unique-password
set ADMIN_NAME=Orvexa Super Admin
set CONFIRM_PRODUCTION_BOOTSTRAP=yes
npm run admin:bootstrap
```

You want HTTP 201. Then immediately:

1. Render production → Environment → **delete** `ADMIN_BOOTSTRAP_KEY` → Save.
2. Confirm a second bootstrap attempt returns 409 or 403.

> Do not run the seeder. Do not create `admin@orvexatech.com` / `admin@password`. Those are demo leftovers.

8.5 First human sign-in

1. Open `https://admin.orvexacloud.in`.
2. Complete the Cloudflare email PIN (inbox of the allowed staff address).
3. Sign in with the email and password from 8.4.
4. You should land on the Super Admin overview. There are **zero** demo stores.

8.6 First real merchant

1. Either approve a marketing-site application or use **Provision New Store** for a real brand.
2. Open the storefront URL the dashboard prints. Confirm it is not Lunar demo catalog unless that brand is a real customer.
3. Seller signs in at `https://app.orvexacloud.in` (or the store dashboard link) with the password you set / the reset email — not a published demo password.

8.7 If anything is wrong

1. Render → the production service → **Manual Rollback** to the previous Live instance (fastest for API).
2. Cloudflare → Workers / Pages → the previous deployment → **Rollback**.
3. Or Actions → **Rollback** → target `production` → last known-good SHA.
4. Tell the owner you have reverted. Do not keep shipping fixes on live without a new PR.

- ☐ Health commit = `main` merge SHA
- ☐ Bootstrap 201, then key deleted
- ☐ PIN + admin password work
- ☐ No demo tenants in the store list
- ☐ Owner agrees “we are live”

Appendix A — Secret worksheet

Print two copies. Fill by hand. Do not type real values into a shared doc.

Staging

Item	Stored in password manager as	Pasted into
Atlas master user password		Render staging `MONGODB_MASTER_URI`
Atlas tenant user password		Render staging `MONGODB_TENANT_BASE_URI`
`JWT_SECRET`		Render staging only
`JWT_REFRESH_SECRET`		Render staging only
`ADMIN_BOOTSTRAP_KEY`		Render staging (delete after use)
`SIGNUP_SIGNING_SECRETS`		Render + website connector
Render deploy hook		GitHub env `staging`
Cloudflare API token / account id		GitHub env `staging`

Production (different values)

Item	Stored in password manager as	Pasted into
Atlas master password		Render production
Atlas tenant password		Render production
`JWT_SECRET`		Render production only
`JWT_REFRESH_SECRET`		Render production only
`ADMIN_BOOTSTRAP_KEY`		Render production — delete same day
`SIGNUP_SIGNING_SECRETS`		Render + website connector
Resend API key + `MAIL_FROM`		Render production
Razorpay live key / secret / webhook		Render production
Render deploy hook		GitHub env `production`

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Appendix B — Production DNS (copy into Cloudflare)

Name	Type	Content	Proxy	Worker
`api`	CNAME	`orvexa-api-production.onrender.com` (use the real hostname)	On	None
`admin`	CNAME	Pages project `orvexa-admin`	On	None
`app`	CNAME	Pages project `orvexa-app`	On	None
`media`	CNAME	R2 bucket `orvexa-media`	On	None
`*`	AAAA	`100::`	On	`orvexa-shop-router`
Merchant custom domain	CNAME or Custom Hostname	`100::` / shop origin	On	`orvexa-shop-router`

URLs after cutover

URL	What you should see
`https://api.orvexacloud.in/api/health`	`{"ok":true,"commit":"…"}`
`https://admin.orvexacloud.in`	Cloudflare PIN, then Orvexa admin sign-in
`https://app.orvexacloud.in`	Seller portal
`https://<slug>.orvexacloud.in`	That merchant's storefront
`https://media.orvexacloud.in/…`	Product image

Appendix C — Three rollback options

When	Where	What you click
API is 500 / health is wrong, frontends are fine	Render → production service	**Manual Rollback** to the previous instance. Fastest.
Admin / app / shop look broken, API health is fine	Cloudflare → Pages / Workers	Previous deployment → **Rollback**
You want API + edge on the same old SHA	GitHub → Actions → **Rollback**	Target `production`, paste the last green SHA, approve the environment

> After any rollback, run `curl https://api.orvexacloud.in/api/health` and confirm the `commit` you expected. Then tell the owner.

Appendix D — Troubleshooting

What you see	What to do
Health 503, `ok: false`	Atlas cannot see Render. Compare outbound IPs to Network Access. Confirm users and URIs. No `0.0.0.0/0` "fix".
Health 200 but `commit` is `unknown` or an old SHA	The hook was called without `&ref=<sha>`, or Auto-Deploy built the wrong commit. Re-run Deploy with the hook; do not type SHAs by hand in Render unless you know the box.
Admin page with no PIN	Access application missing or attached to the wrong hostname.
CORS errors in the seller browser	Add that exact origin to Render `CORS_ORIGIN`. Custom merchant domains are picked up from the database within a minute.
Storefront 404 on `brand.orvexacloud.in`	Wildcard AAAA + shop-router route missing, or the slug is reserved (`admin`, `api`, `media`, …).
Bootstrap 403	Key too short, placeholder key, or already used. Key must be ≥ 16 random characters and not `orvexa_bootstrap_2026`.
Bootstrap 409	An admin already exists. Use the admin UI; do not seed.
Actions cannot deploy Pages	Token missing Pages Edit; project name typo; account id wrong.
Gitleaks fails on a docs file	Do not disable the scanner. Move the secret out of git. `.gitleaks.toml` already ignores `docs/` and `.env.example`.

Appendix E — Cost reminder

Item	Monthly (approx.)
Atlas M10 dedicated, AWS Singapore	~$58
Render Starter (one production API; staging is extra if you keep it)	$7 each
Cloudflare Workers Paid	~$5
GitHub Team / Render seats	per user, not in the $70–80 foundation

© 2026 Orvexa Tech Private Limited · Operator setup checklist v1.0 · Do not store real secrets in this file